🎓 Fiche de Révision : TP Noté - Machine Learning (ACID)

Objectif : Maîtriser le pipeline complet : Chargement $\rightarrow$ Classification $\rightarrow$ Évaluation $\rightarrow$ Optimisation.

---

1. Préparation des Données (Le B.A.-BA)

Avant tout modèle, il faut préparer le terrain. Si cette étape est ratée, le reste ne fonctionnera pas.

A. Chargement et Nettoyage

- **Bibliothèque :** `pandas`
- **Commande clé :** `pd.read_csv('fichier.csv', sep=';', decimal=',')`
  - *Attention :* Vérifie toujours le séparateur (`;` ou `,`) et le format des décimales.
- **Vérification :**
  - `df.head()` : Voir les 5 premières lignes.
  - `df.info()` : Vérifier s'il y a des valeurs nulles (`NaN`).

B. Séparation (Features / Target)

On sépare ce qui sert à prédire (X) de ce qu'on veut deviner (y).

```
X = df.drop('classe', axis=1) # Les caractéristiques
y = df['classe']              # La cible
```

C. Train / Test Split (Crucial)

On coupe les données pour ne jamais tester sur ce qu'on a appris.

```
from sklearn.model_selection import train_test_split
# 80% pour apprendre, 20% pour tester
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

---

2. Les Classifieurs (Algorithmes)

A. Naive Bayes (GaussianNB)

- **C'est quoi ?** Un algo probabiliste basé sur le théorème de Bayes.
- **Hypothèse :** Il suppose que les données suivent une loi Normale (Gaussienne).
- **Avantage :** Très rapide, simple, marche bien même avec peu de données.
- **Code :**

```
from sklearn.naive_bayes import GaussianNB
nb = GaussianNB()
nb.fit(X_train, y_train)
```

B. k-Nearest Neighbors (k-NN)

- **C'est quoi ?** Il regarde les k voisins les plus proches d'un point et vote pour la classe majoritaire.
- **Paramètre clé (k) :**
  - k trop petit (ex: 1) = Sensible au bruit (sur-apprentissage).
  - k trop grand = Trop lisse (sous-apprentissage).
- **Code :**

```
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
```

---

3. Évaluation (La preuve que ça marche)

Ne jamais se fier uniquement à l'accuracy (taux de réussite), surtout si les classes sont déséquilibrées.

La Matrice de Confusion

C'est le tableau de bord des erreurs.

- **Diagonale :** Ce sont les bonnes réponses (Vrai Positif, Vrai Négatif).
- **Le reste :** Ce sont les erreurs (Faux Positifs, Faux Négatifs).

```
from sklearn.metrics import confusion_matrix, accuracy_score
y_pred = model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print("Accuracy :", accuracy_score(y_test, y_pred))
```

---

4. Réduction de Dimension (Optimisation)

Quand on a trop de colonnes (dimensions), le calcul est lent et la performance peut baisser ("Malédiction de la dimension").

A. ACP (Analyse en Composantes Principales)

- **Type :** Non Supervisé (ne regarde pas les classes y).
- **Principe :** Compresser les données en gardant le max de variance (d'information).
- **Obligatoire :** Il faut **centrer-réduire** les données avant (`StandardScaler`).
- **Critère du Coude :** Regarder le graphique des valeurs propres pour choisir le nombre d'axes.

B. LDA (Analyse Discriminante Linéaire)

- **Type :** Supervisé (utilise les classes y).
- **Principe :** Chercher les axes qui **séparent** le mieux les groupes (classes).
- **Limite :** Le nombre de dimensions max est égal au (Nombre de classes - 1).

---

5. Le "Code Miracle" (Cheat Sheet)

Voici le bloc de code à copier-coller et adapter demain si tu dois chercher le meilleur k pour les k-NN (question classique du TP 2024).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

# Recherche du meilleur k
scores = []
k_range = range(1, 20) # On teste k de 1 à 19

for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train, y_train)
    y_pred = knn.predict(X_test)
    scores.append(accuracy_score(y_test, y_pred))

# Affichage graphique
plt.plot(k_range, scores, marker='o')
plt.xlabel('Valeur de k')
plt.ylabel('Accuracy')
plt.title('Meilleur k pour k-NN')
plt.grid()
plt.show()

# Trouver le k optimal automatiquement
best_k = k_range[np.argmax(scores)]
print(f"Le meilleur k est : {best_k}")
```